



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE302L- Database Systems

Register Number	24BCE1731	Subject Code	BCSE302L
Name of the Student	Abhinav Bhuniya	Subject Name	Database Systems
Programme	BTech CSE	DA-2	
Date	08/04/2026		

School: SCOPE

Project Title: Inventory & Supplier Management System

Course Code: BCSE302L

Course Title: Database Management Systems

Class Number: CH2025260501233

Semester: Winter 2025-26

Slot: A1+TA1

Team members: Abhinav Bhuniya, 24BCE1731

Aveek Sanyal, 24BCE1081

Gaurav Satyedev, 24BCE1828

Course Faculty: Dr. L.M. Jenila Livingston

1. INTRODUCTION

This project is an **Inventory & Supplier Management System** developed to manage products, suppliers, purchase orders, stock, and payments.

The system demonstrates:

- CRUD operations
- Relational database design
- Use of foreign keys
- JOIN queries for data retrieval

The database schema was designed using ER modeling and then converted into relational tables. This assignment focuses on **normalization and backend schema design**.

2. LIST OF RELATIONS (FROM ER MAPPING)

1. **PRODUCT**(product_id, product_name, category, unit_price)
2. **SUPPLIER**(supplier_id, supplier_name, phone, email)
3. **PURCHASE_ORDER**(order_id, product_id, supplier_id, quantity, order_date)
4. **STOCK**(stock_id, product_id, available_quantity, last_updated)
5. **PAYMENT**(payment_id, order_id, amount, payment_date, payment_mode)

3. NORMALIZATION PROCESS

3.1 PRODUCT TABLE

Original Table:

PRODUCT(product_id, product_name, category, unit_price)

Functional Dependency:

product_id → product_name, category, unit_price

Candidate Key:

product_id

1NF:

- All attributes are atomic
- No repeating groups
- ✓ Satisfies 1NF

2NF:

- No composite key

- No partial dependency
- ✓ Satisfies 2NF

3NF:

- No transitive dependency
- ✓ Satisfies 3NF

BCNF:

- Determinant is a super key
- ✓ Satisfies BCNF

Justification:

All attributes depend fully on the primary key. No redundancy or anomalies exist.

3.2 SUPPLIER TABLE

Original Table:

SUPPLIER(supplier_id, supplier_name, phone, email)

Functional Dependency:

supplier_id $\rightarrow$ supplier_name, phone, email

Candidate Key:

supplier_id

Normalization:

✓ Already in 1NF, 2NF, 3NF, and BCNF

Justification:

All attributes depend on a single primary key with no transitive or partial dependencies.

3.3 PURCHASE_ORDER TABLE

Original Table:

PURCHASE_ORDER(order_id, product_id, supplier_id, quantity, order_date)

Functional Dependency:

order_id $\rightarrow$ product_id, supplier_id, quantity, order_date

Candidate Key:

order_id

Normalization:

- ✓ 1NF: Atomic attributes
- ✓ 2NF: No composite key
- ✓ 3NF: No transitive dependency
- ✓ BCNF: Determinant is a super key

Justification:

Each order is uniquely identified by order_id and contains no redundant dependencies.

3.4 STOCK TABLE

Original Table:

STOCK(stock_id, product_id, available_quantity, last_updated)

Functional Dependency:

stock_id → product_id, available_quantity, last_updated

Candidate Key:

stock_id

Normalization:

- ✓ Already in BCNF

Justification:

All attributes depend directly on the primary key without redundancy.

3.5 PAYMENT TABLE

Original Table:

PAYMENT(payment_id, order_id, amount, payment_date, payment_mode)

Functional Dependency:

payment_id → order_id, amount, payment_date, payment_mode

Candidate Key:

payment_id

Normalization:

- ✓ Already in BCNF

Justification:

No partial or transitive dependencies exist.

4. FINAL RESULT OF NORMALIZATION

All tables are already in **Boyce-Codd Normal Form (BCNF)**.

No further decomposition was required.

5. FINAL DATABASE SCHEMA (BCNF)

PRODUCT(product_id PK, product_name, category, unit_price)

SUPPLIER(supplier_id PK, supplier_name, phone, email)

PURCHASE_ORDER(order_id PK, product_id FK, supplier_id FK, quantity, order_date)

STOCK(stock_id PK, product_id FK, available_quantity, last_updated)

PAYMENT(payment_id PK, order_id FK, amount, payment_date, payment_mode)

6. CONSTRAINTS

Primary Keys:

- product_id
- supplier_id
- order_id
- stock_id
- payment_id

Foreign Keys:

- product_id → PRODUCT(product_id)
- supplier_id → SUPPLIER(supplier_id)
- order_id → PURCHASE_ORDER(order_id)

7. RELATIONSHIPS

- One Product → Many Purchase Orders
- One Supplier → Many Purchase Orders
- One Product → One Stock Record
- One Order → One Payment

8. BACKEND IMPLEMENTATION (SQL)

```
CREATE TABLE product (  
  product_id SERIAL PRIMARY KEY,  
  product_name VARCHAR(100),  
  category VARCHAR(50),  
  unit_price DECIMAL(10,2)
```

```
);
```

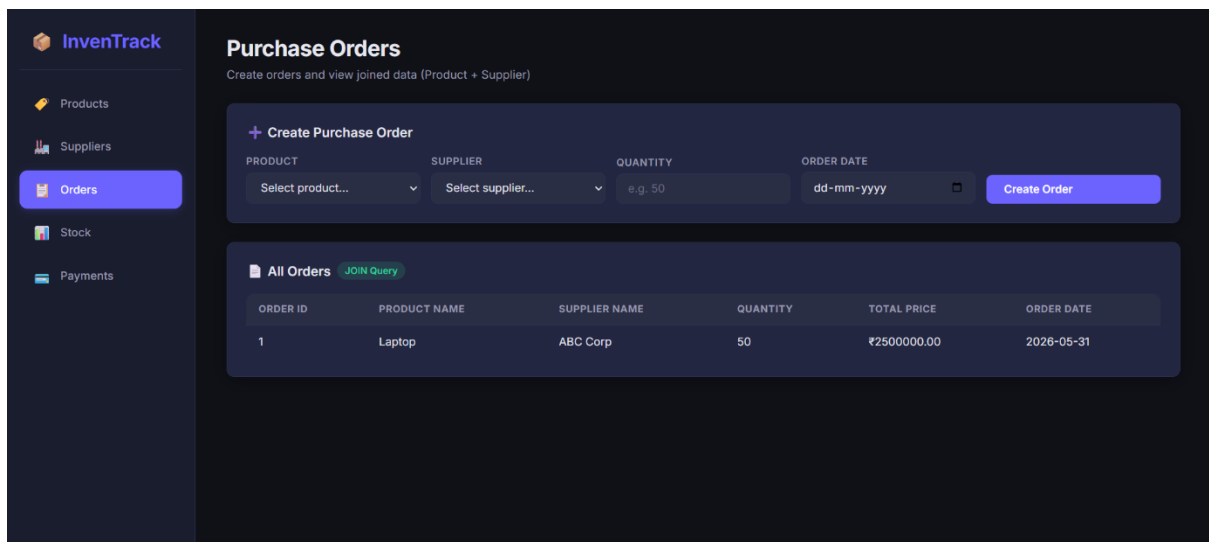
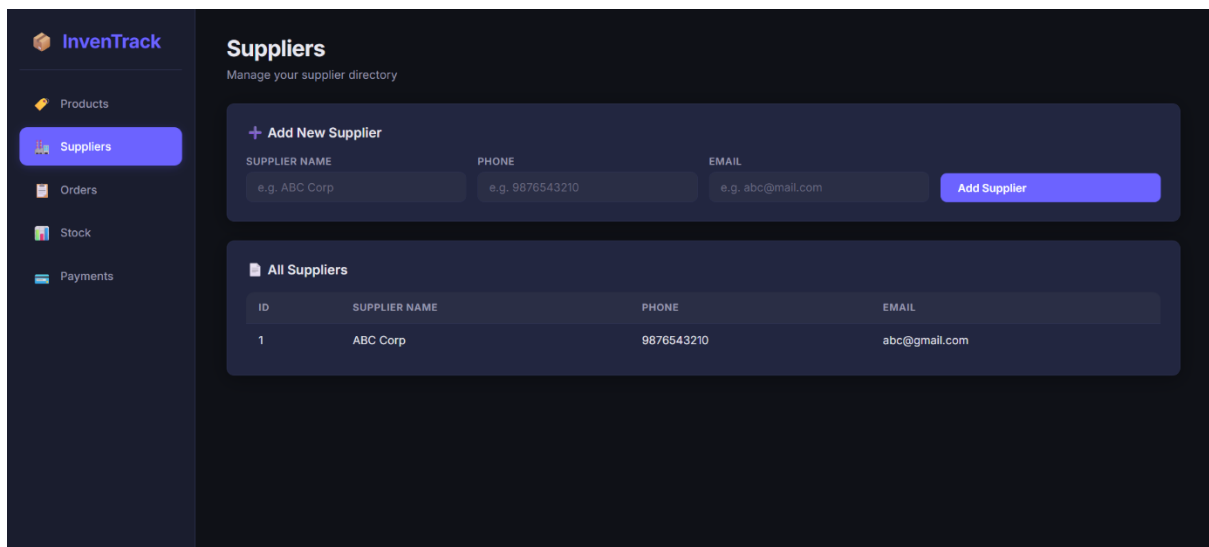
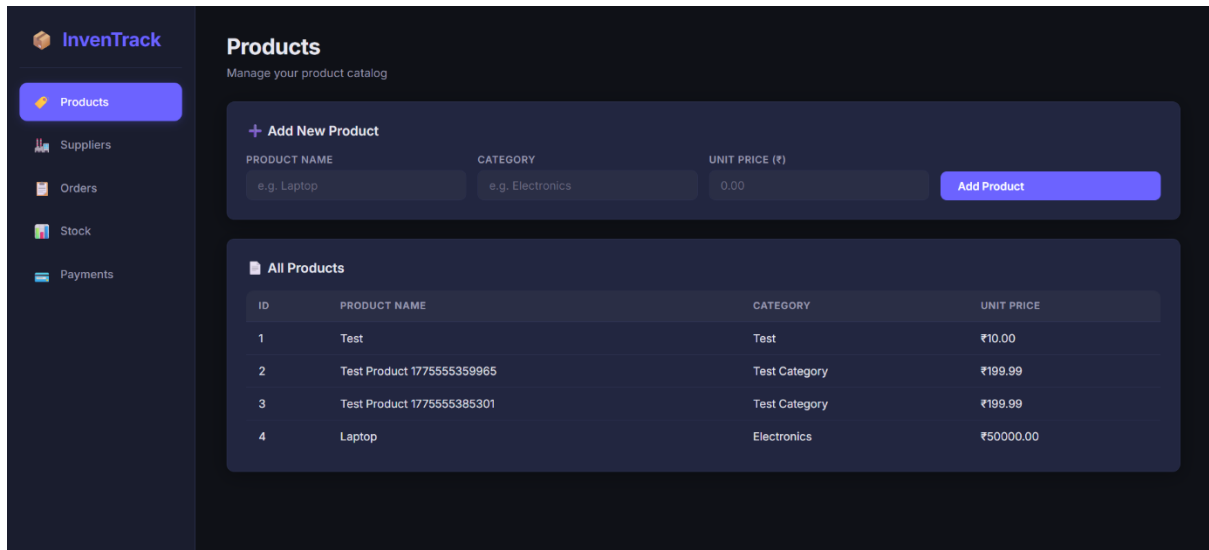
```
CREATE TABLE supplier (  
    supplier_id SERIAL PRIMARY KEY,  
    supplier_name VARCHAR(100),  
    phone VARCHAR(15),  
    email VARCHAR(100)  
);
```

```
CREATE TABLE purchase_order (  
    order_id SERIAL PRIMARY KEY,  
    product_id INT,  
    supplier_id INT,  
    quantity INT,  
    order_date DATE,  
    FOREIGN KEY (product_id) REFERENCES product(product_id),  
    FOREIGN KEY (supplier_id) REFERENCES supplier(supplier_id)  
);
```

```
CREATE TABLE stock (  
    stock_id SERIAL PRIMARY KEY,  
    product_id INT,  
    available_quantity INT,  
    last_updated TIMESTAMP,  
    FOREIGN KEY (product_id) REFERENCES product(product_id)  
);
```

```
CREATE TABLE payment (  
    payment_id SERIAL PRIMARY KEY,  
    order_id INT,  
    amount DECIMAL(10,2),  
    payment_date DATE,  
    payment_mode VARCHAR(20),  
    FOREIGN KEY (order_id) REFERENCES purchase_order(order_id)  
);
```

9. UI Screenshots



InvenTrack

Products

Suppliers

Orders

Stock

Payments

Stock

Initialize product stock and update available quantities

+ Initialize Stock

PRODUCT

Select product...

INITIAL QUANTITY

e.g. 100

Add Stock Record

Stock Levels

STOCK ID	PRODUCT NAME	AVAILABLE QTY	LAST UPDATED	ACTION
1	Laptop	97	4/7/2026	Update

InvenTrack

Products

Suppliers

Orders

Stock

Payments

Payments

View and manage payment records

+ Record Payment

ORDER ID

Select order...

AMOUNT (₹)

0.00

PAYMENT DATE

dd-mm-yyyy

PAYMENT MODE

Select mode...

Add Payment

All Payments

PAYMENT ID	ORDER ID	AMOUNT	PAYMENT DATE	MODE	ACTION
No payments yet					

10. CONCLUSION

The database was successfully normalized up to **BCNF**, ensuring:

- Elimination of redundancy
- Prevention of update, insertion, and deletion anomalies
- Improved data consistency and integrity

The backend schema is clean, efficient, and ready for implementation, as demonstrated through the working system